

4주차

DML 기초와 트랜잭션

INSERT·UPDATE·DELETE, 시드 적재, COMMIT과 ROLLBACK

한국기술교육대학교 경영학부

경영데이터베이스및실습 (CCT770) · 2026학년도 2학기

이번 주를 마치면

목표 ①	INSERT로 단건·다건·조회 결과 삽입을 수행한다
목표 ②	UPDATE·DELETE에서 WHERE의 역할을 알고 안전하게 수정·삭제한다
목표 ③	DELETE·TRUNCATE·DROP의 차이를 설명한다
목표 ④	ACID 특성을 이해하고 COMMIT·ROLLBACK으로 제어한다
목표 ⑤	주문 취소·재고 복구 같은 업무를 하나의 트랜잭션으로 묶는다

지난주 DDL로 만든 빈 그릇에 다운유통 데이터를 채웁니다

오늘 다루는 다섯 가지

1 INSERT — 데이터 삽입

2 다운유통 시드 데이터 적재 (핵심 실습)

3 UPDATE — 데이터 수정

4 DELETE / TRUNCATE / DROP

5 트랜잭션 — 전부 성공 아니면 전부 취소

1. INSERT — 데이터 삽입

INSERT 쓰는 법 ①~③ — 컬럼 목록이 핵심

```
1  -- ① 컬럼 지정 (권장): 테이블 구조가 바뀌어도 안전
2  INSERT INTO customers (cust_id, name, grade, city, email, joined_date, points)
3  VALUES ('C011', '강현우', 'SILVER', '서울', 'hyunwoo@example.com', '2026-07-01', 0);
4
5  -- ② 컬럼 생략: 반드시 테이블 정의 순서대로 모든 값을 나열
6  INSERT INTO customers
7  VALUES ('C012', '노은채', 'SILVER', '대구', 'eunchae@example.com', '2026-07-03', 0);
8
9  -- ③ 일부 컬럼만 지정: 나머지는 DEFAULT 값 또는 NULL이 들어간다
10 INSERT INTO customers (cust_id, name, joined_date)
11 VALUES ('C013', '유지호', '2026-07-05');
```

1

2-3행

1

컬럼 7개 ↔ 값 7개

순서대로 1:1 대응, 문자열·날짜는 작은따옴표

2

6-7행

2

목록 생략

DESC 순서 그대로 7개 — 하나라도 빠지면 오류

3

10-11행

3

세 컬럼만

grade·points는 DEFAULT, city·email은 NULL

기본 골격: INSERT INTO 테이블 (컬럼목록) VALUES (값목록);

INSERT 쓰는 법 ④·⑤ — 여러 행, 조회 결과

```
1 -- ④ 여러 행 한 번에 (성능상 유리)
2 INSERT INTO products (prod_id, name, category, price, stock) VALUES
3 ('P011', '스테인리스 텀블러', '생활용품', 15000, 40),
4 ('P012', '형광펜 5색 세트', '문구', 5500, 150);
5
6 -- ⑤ 다른 테이블의 조회 결과를 삽입 (INSERT ... SELECT)
7 CREATE TABLE vip_customers LIKE customers;
8 INSERT INTO vip_customers
9 SELECT * FROM customers WHERE grade = 'VIP';
```

1

3-4행

1 (행), (행)

쉼표로 이어 붙이고 마지막 행은 세미콜론

2

7행

2 구조만 복사

LIKE — 데이터는 아직 0행

3

8-9행

3 VALUES 자리에 SELECT

VIP 고객 전체 컬럼 — 컬럼 수·순서가 같아야 한다

vip_customers는 분석용 사본 — 4절의 삭제 명령 실습에 다시 쓴다

1. INSERT — 데이터 삽입

제약조건은 INSERT 시점에 작동한다

■ 증상

! **ERROR 1062 (Duplicate entry)**

! **ERROR 1452**

! **같은 키가 있으면 갱신하고 싶다**

■ 원인

→ 이미 등록된 이메일로 다시 넣었다 — UNIQUE가 막는다

→ 존재하지 않는 고객번호로 주문을 넣었다 — 외래키가 막는다

→ INSERT는 새 행만 넣는다

■ 해결

✓ **중복을 무시하려면 INSERT IGNORE**

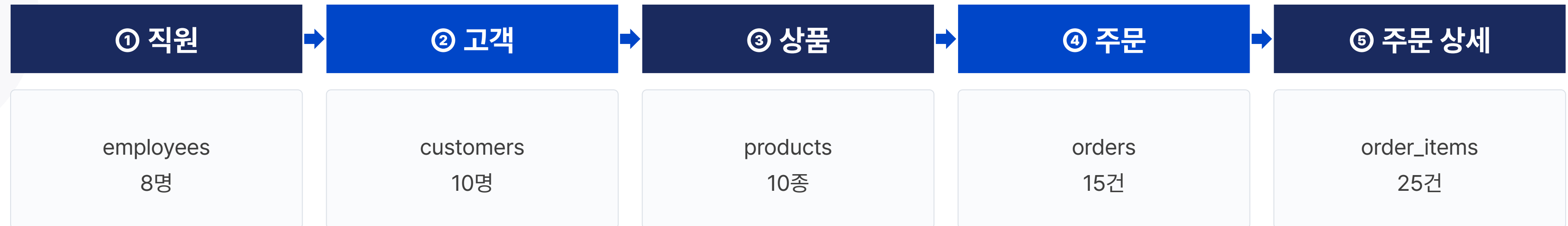
✓ **부모 행(고객)을 먼저 넣는다**

✓ **INSERT ... ON DUPLICATE KEY UPDATE (MySQL 전용)**

오류 번호를 읽으면 어느 제약이 막았는지 바로 알 수 있다

적재 순서 = 외래키 순서

● 핵심 실습



부모(직원·고객·상품)를 먼저, 자식(주문 → 주문 상세)을 나중에 — 순서를 바꾸면 ERROR 1452

앞으로 모든 주차의 예제·퀴즈·시험은 이 데이터로 답이 계산된다 — 반드시 그대로 실행

① 직원 8명 — 팀장 3명, E008은 연봉 NULL

```
1  -- 값 순서: emp_id, name, dept, position, hire_date, salary(만원), manager_id
2  INSERT INTO employees VALUES
3    ('E001', '김성호', '영업팀', '팀장', '2018-03-05', 520, NULL),
4    ('E002', '이지은', '영업팀', '대리', '2021-07-01', 380, 'E001'),
5    ('E003', '박준영', '영업팀', '사원', '2024-01-15', 310, 'E001'),
6    ('E004', '최민서', '마케팅팀', '팀장', '2019-09-02', 500, NULL),
7    ('E005', '정다은', '마케팅팀', '사원', '2023-05-10', 320, 'E004'),
8    ('E006', '강태윤', '물류팀', '팀장', '2020-02-17', 480, NULL),
9    ('E007', '윤서준', '물류팀', '사원', '2025-03-03', 300, 'E006'),
10   ('E008', '조하늘', '경영지원팀', '사원', '2026-01-05', NULL, NULL);
```

manager_id는 자기참조 FK — 상사(E001·E004·E006)를 먼저 넣어야 부하 행이 들어간다

② 고객 10명 — 미입력 칸이 있는 고객

```
1  -- 값 순서: cust_id, name, grade, city, email, joined_date, points
2  INSERT INTO customers VALUES
3    ('C001', '한가람', 'VIP', '서울', 'garam@example.com', '2024-03-12', 4500),
4    ('C002', '오세영', 'GOLD', '부산', 'seyoung@example.com', '2024-06-20', 2300),
5    ('C003', '김도현', 'SILVER', '서울', 'dohyun@example.com', '2025-01-08', 800),
6    ('C004', '임수빈', 'VIP', '대전', 'subin@example.com', '2023-11-02', 5200),
7    ('C005', '박서연', 'SILVER', '광주', NULL, '2025-04-17', 300),
8    ('C006', '장민혁', 'GOLD', '서울', 'minhyuk@example.com', '2024-09-30', 1900),
9    ('C007', '문예린', 'SILVER', NULL, 'yerin@example.com', '2025-08-25', 450),
10   ('C008', '신우진', 'SILVER', '부산', 'woojin@example.com', '2026-02-14', 150),
11   ('C009', '배소율', 'GOLD', '인천', 'soyul@example.com', '2025-06-11', 2100),
12   ('C010', '서지안', 'SILVER', '대전', 'jian@example.com', '2026-03-01', 0);
```

UNIQUE 컬럼이라도 NULL은 여러 행 허용 · C010은 주문이 한 건도 없는 고객

③ 상품 10종 — P004 품절, P010 주문 0건

```
1  -- 값 순서: prod_id, name, category, price(원), stock(재고 수량)
2  INSERT INTO products VALUES
3    ('P001', '무선 전기포트', '가전', 42000, 25),
4    ('P002', '공기청정기', '가전', 189000, 8),
5    ('P003', '유기농 원두 500g', '식품', 18500, 60),
6    ('P004', '제주 감귤 3kg', '식품', 24000, 0),
7    ('P005', '곡물 시리얼 세트', '식품', 15800, 42),
8    ('P006', '대나무 칫솔 4입', '생활용품', 6900, 120),
9    ('P007', '천연 세탁세제', '생활용품', 12500, 75),
10   ('P008', '접이식 장바구니', '생활용품', 9800, 50),
11   ('P009', '3색 볼펜 세트', '문구', 4500, 200),
12   ('P010', 'A5 스프링 노트', '문구', 3200, 180);
```

P004 → 발주 대상 조회 실습 · P010 → 비인기 상품 찾기 실습

④ 주문 15건 — 여기만 컬럼을 명시한다

```
1  INSERT INTO orders (order_id, cust_id, emp_id, order_date, status) VALUES
2  (1001, 'C001', 'E002', '2026-01-10', '배송완료'),
3  (1002, 'C002', NULL, '2026-01-22', '배송완료'),
4  (1003, 'C004', 'E002', '2026-02-03', '배송완료'),
5  (1004, 'C001', NULL, '2026-02-14', '배송완료'),
6  (1005, 'C006', 'E003', '2026-02-27', '취소'),
7  (1006, 'C003', NULL, '2026-03-05', '배송완료'),
8  (1007, 'C009', 'E002', '2026-03-18', '배송완료'),
9  (1008, 'C004', NULL, '2026-03-29', '배송완료'),
10 (1009, 'C008', 'E003', '2026-04-08', '배송완료'),
11 (1010, 'C001', NULL, '2026-04-21', '배송완료'),
12 (1011, 'C006', 'E002', '2026-05-02', '배송중'),
13 (1012, 'C002', NULL, '2026-05-16', '배송중'),
14 (1013, 'C007', 'E003', '2026-05-27', '취소'),
15 (1014, 'C004', NULL, '2026-06-04', '배송중'),
16 (1015, 'C005', 'E002', '2026-06-10', '주문접수');
```

order_id는 AUTO_INCREMENT지만 정답 계산이 흔들리지 않게 번호를 직접 지정 · emp_id NULL 7건 = 온라인 주문

⑤ 주문 상세 25건 — 복합키

```
1  -- 값 순서: order_id, prod_id, qty(수량), unit_price(주문 당시 단가)
2  INSERT INTO order_items VALUES
3    (1001, 'P001', 1, 42000), (1001, 'P003', 2, 18500),
4    (1002, 'P006', 3, 6900), (1002, 'P007', 1, 12500),
5    (1003, 'P002', 1, 189000),
6    (1004, 'P005', 2, 15800), (1004, 'P009', 5, 4500),
7    (1005, 'P004', 2, 24000),
8    (1006, 'P003', 1, 18500), (1006, 'P006', 2, 6900),
9    (1007, 'P007', 2, 12500), (1007, 'P008', 1, 9800),
10   (1008, 'P001', 1, 42000), (1008, 'P004', 3, 24000),
11   (1009, 'P009', 10, 4500),
12   (1010, 'P002', 1, 189000), (1010, 'P003', 2, 18500),
13   (1011, 'P005', 3, 15800),
14   (1012, 'P008', 2, 9800), (1012, 'P006', 1, 6900),
15   (1013, 'P003', 1, 18500),
16   (1014, 'P007', 1, 12500), (1014, 'P009', 3, 4500),
17   (1015, 'P001', 1, 42000), (1015, 'P005', 1, 15800);
```

unit_price를 따로 두는 이유 — products.price가 바뀌어도 주문 당시 금액은 그대로 남아야 한다

적재 결과 확인 — 행 수를 대조한다

테이블	확인 SQL	기대값
직원	SELECT COUNT(*) FROM employees;	8
고객	SELECT COUNT(*) FROM customers;	10
상품	SELECT COUNT(*) FROM products;	10
주문	SELECT COUNT(*) FROM orders;	15
주문 상세	SELECT COUNT(*) FROM order_items;	25

COUNT(*)는 NULL이 있는 행도 센다 — 숫자가 다르면 어느 INSERT가 오류로 멈췄는지 결과 창 메시지를 다시 본다

시드에 일부러 심어 둔 데이터

데이터	특징	쓰이는 곳
E008 조하늘	salary NULL + manager_id NULL	5주차 IS NULL 실습
C005 박서연 · C007 문예린	email NULL · city NULL	C007은 도시별 집계에서 빠진다
C010 서지안	주문 0건	7주차 LEFT JOIN · 미주문 고객 찾기
P004 · P010	재고 0(품절) · 주문 0건	발주 대상 조회 · 비인기 상품 찾기
주문 1005 · 1013	status '취소'	매출 집계 때 WHERE로 반드시 제외
emp_id NULL 7건	담당 직원 없는 온라인 주문	= NULL이 아니라 IS NULL로 찾는다

NULL·취소·미주문 데이터는 앞으로 NULL 처리와 조인 실습에서 계속 등장한다

3. UPDATE — 데이터 수정

SET은 무엇을 어떻게, WHERE는 어느 행을

```
1  -- 단일 컬럼 수정: 상품 가격 조정
2  UPDATE products SET price = 45000 WHERE prod_id = 'P001';
3
4  -- 여러 컬럼 동시 수정: 등급 승격 + 축하 포인트 지급
5  UPDATE customers
6  SET    grade  = 'GOLD',
7        points = points + 1000
8  WHERE cust_id = 'C003';
9
10 -- 계산식 수정: 주문이 확정되면 재고를 차감
11 UPDATE products SET stock = stock - 1 WHERE prod_id = 'P001';
```

1

2행

1

기본키로 1행만

WHERE prod_id = 'P001'

2

6-7행

2

쉽표로 나열

오른쪽 points는 바꾸기 전 값 — 현재에 1000을 더한다

3

11행

3

숫자 대신 식

지금 재고가 몇 개든 안전하게 1 줄어든다

기본 골격: UPDATE 테이블 SET 컬럼 = 값 WHERE 조건; — WHERE가 없으면 전체 행이 바뀐다

CASE와 결합 — 전 고객 등급 일괄 재산정

```
1 UPDATE customers
2 SET   grade = CASE WHEN points >= 4000 THEN 'VIP'
3         WHEN points >= 1500 THEN 'GOLD'
4         ELSE 'SILVER' END;
```

1

1행

1 **의도된 일괄 수정**

이런 경우가 아니면 WHERE는 필수다

2

2-4행

2 **위에서부터 검사**

4000 이상 VIP → 1500 이상 GOLD → 나머지 SILVER

실행 전에 `SELECT name, points, CASE ... END FROM customers;` 로 바뀔 등급을 먼저 눈으로 확인한다

WHERE를 빠뜨리면 전 고객이 바뀐다

■ 증상

! UPDATE customers SET grade = 'VIP';

! 어느 행이 바뀔지 모른다

! Error 1175 (Safe Updates 모드)

■ 원인

→ WHERE가 없어 고객 10명 전원이 대상

→ 조건을 머릿속으로만 확인했다

→ 기본키 조건이 없는 UPDATE·DELETE를 Workbench가 막는다

■ 해결

✓ 할인·적립 정책이 통째로 무너진다 — 반드시 조건을 붙인다

✓ 같은 WHERE 조건으로 SELECT를 먼저 실행한다

✓ 실습 자료실 2.4대로 끄거나 SET SQL_SAFE_UPDATES = 0;

Safe Updates는 안전장치이지만 WHERE name = '...' 같은 정상 문장도 막는다

4. DELETE / TRUNCATE / DROP

지운다는 명령 셋 — 행, 전부, 테이블

```
1  -- DELETE: 조건에 맞는 행만 삭제 (DML, 롤백 가능)
2  DELETE FROM vip_customers WHERE points < 5000;
3
4  -- TRUNCATE: 전체 행을 즉시 제거 (DDL, 롤백 불가, AUTO_INCREMENT 초기화)
5  TRUNCATE TABLE vip_customers;
6
7  -- DROP: 테이블 구조 자체를 삭제
8  DROP TABLE vip_customers;
```

2행

1 조건 삭제

WHERE를 빼면 전체 행 — UPDATE와 같은 주의

5행

2 빈 그릇으로

WHERE 불가, 구조(컬럼·제약)는 남는다

8행

3 통째로 제거

컬럼 정의·제약조건까지 사라진다

세 문장 모두 분석용 사본 vip_customers 대상 — 원본 customers는 그대로다

세 명령 한눈에 비교

구분	DELETE	TRUNCATE	DROP
분류	DML	DDL	DDL
WHERE 조건	사용 가능	불가 (전체 삭제)	해당 없음
롤백	가능	불가 (자동 커밋)	불가
속도	느림 (행 단위 로그)	빠름	빠름
AUTO_INCREMENT	유지	초기화	테이블과 함께 제거
테이블 구조	유지	유지	삭제

주문 취소는 DELETE가 아니라 UPDATE

01 시드의 1005·1013

지워지지 않고

status '취소'로 남아 있다

02 지우면 사라지는 것

취소율 분석도

고객 문의 대응 근거도

03 기록성 데이터

지우지 않고

상태만 바꾼다

취소 기록이 곧 분석·CS 근거다

주문 취소 = 두 문장이 한 몸

① 장부 — 상태 변경

01

주문 상태를 '취소'로 바꾼다
(기록은 지우지 않는다)

② 창고 — 재고 복구

02

주문 때 차감했던 재고를
되돌린다

트랜잭션 = 하나의 논리적 작업 단위 — 모두 성공하거나 모두 실패, 한쪽만 반영되면 장부와 창고가 어긋난다

트랜잭션이 지키는 네 가지 성질

특성	영문	의미	보장 기법
원자성	Atomicity	전부 수행되거나 전혀 수행되지 않음 (All or Nothing)	UNDO 로그, ROLLBACK
일관성	Consistency	트랜잭션 전후로 무결성 제약이 유지됨	무결성 제약조건 검사
고립성	Isolation	동시에 실행되는 트랜잭션끼리 간섭하지 않음	락(Lock), MVCC
지속성	Durability	커밋된 결과는 장애가 나도 영구히 보존됨	REDO 로그, 체크포인트

5.2 COMMIT / ROLLBACK

주문 취소 처리를 한 단위로 묶기

```
1 SELECT @@autocommit;      -- MySQL 기본값 1 (문장마다 자동 확정)
2 SET autocommit = 0;
3
4 START TRANSACTION;
5 UPDATE orders SET status = '취소' WHERE order_id = 1005;
6 UPDATE products SET stock = stock + 2 WHERE prod_id = 'P004';
7
8 COMMIT;                    -- 확정 (되돌릴 수 없음)
9 -- ROLLBACK;              -- 취소 (START TRANSACTION 시점으로 복원)
```

1

2행

1 자동 커밋 끄기

COMMIT을 직접 할 때까지 확정되지 않는다

2

3

4행

2 트랜잭션 경계

또는 BEGIN — COMMIT/ROLLBACK까지가 한 단위

4

5-6행

3 장부 + 창고

주문 1005에는 P004 제주 감귤이 2개 담겨 있었다

4

8-9행

4 둘 중 하나로 끝낸다

의도와 맞으면 COMMIT, 이상하면 ROLLBACK

SAVEPOINT — 부분 취소 지점

```
1 START TRANSACTION;
2 UPDATE customers SET points = points + 500 WHERE cust_id = 'C001';
3 SAVEPOINT sp1;
4 UPDATE customers SET points = points + 500 WHERE cust_id = 'C002';
5 ROLLBACK TO SAVEPOINT sp1;  -- C002 적립만 취소, C001 적립은 유지
6 COMMIT;                  -- C001 적립만 최종 반영
```

2행

- 1 ① C001 한가람 500점
트랜잭션 시작 뒤 첫 적립

3행

- 2 되돌림 기준점 sp1
여기까지는 지키고 싶다

4-6행

- 3 ② C002만 취소 후 확정
sp1 이후만 되돌리고, COMMIT이 따로 필요하다

트랜잭션으로 묶어야 하는 업무

01

주문 접수

orders.order_items 삽입
+ 재고 차감

02

주문 취소

orders 상태 변경
+ 재고 복구

03

포인트 결제

포인트 차감
+ 주문 등록

04

재고 이동

출고 창고 차감
+ 입고 창고 가산

주의 — DDL(CREATE·ALTER·DROP·TRUNCATE)은 암시적 커밋을 일으켜 진행 중인 트랜잭션을 확정해 버린다

확인 퀴즈

DELETE와 TRUNCATE의 차이로 옳지 않은 것은?

① DELETE는 WHERE 조건을 사용할 수 있다

② TRUNCATE는 롤백할 수 없다

③ TRUNCATE는 AUTO_INCREMENT를 초기화한다

④ DELETE는 테이블 구조까지 삭제한다

정답 및 해설

정답

④ DELETE는 테이블 구조까지 삭제한다

해설

DELETE는 행만 삭제하고 테이블 구조는 유지합니다. 구조까지 삭제하는 것은 DROP입니다.

핵심 요약

핵심 ①	INSERT는 컬럼을 명시하는 방식이 안전하다
핵심 ②	UPDATE·DELETE는 WHERE를 반드시 확인 — 없으면 전체 행
핵심 ③	DELETE(DML·롤백 가능) / TRUNCATE(DDL·초기화) / DROP(구조 삭제)
핵심 ④	주문 같은 기록성 데이터는 지우지 않고 상태를 바꾼다
핵심 ⑤	ACID = 원자성·일관성·고립성·지속성, 한 몸인 업무는 묶는다
핵심 ⑥	DDL은 암시적 커밋 — 트랜잭션 안에서 쓰지 않는다

시드 데이터 적재 결과(8·10·10·15·25)를 꼭 확인해 두세요

다음 주차 예고

5주차 · SELECT 기본

01

SELECT 구조

작성 순서와
실행 순서

02

WHERE 조건

BETWEEN·IN
LIKE 패턴

03

NULL 처리

IS NULL과
3값 논리

04

정렬과 함수

ORDER BY·LIMIT
내장 함수

4주차 마무리

학습 마무리

이번 주 적재한 다운유통 데이터로 다음 주부터 조회를 시작합니다

✓ INSERT 다섯 가지 방식

✓ WHERE를 확인하는 UPDATE·DELETE

✓ 시드 데이터 적재와 행 수 확인

✓ COMMIT·ROLLBACK·SAVEPOINT

이어지는 활동

4주차 실습 10문항 · 퀴즈 3